Инструменты воспроизводимых исследований

DrWatson. Лабораторная работа №4

Федорова А. И.

2026-04-04

Содержание i

1. Вводная часть
2. Теоретическая часть
3. Выполнение работы
4. Результаты

Раздел 1

Вводная часть

Актуальность

- Воспроизводимость — ключевое требование современной науки

Актуальность

- Воспроизводимость — ключевое требование современной науки
- Научный проект требует чёткой структуры и контроля версий данных

Актуальность

- Воспроизводимость — ключевое требование современной науки
- Научный проект требует чёткой структуры и контроля версий данных
- Ручное управление файлами симуляций приводит к ошибкам

Актуальность

- Воспроизводимость — ключевое требование современной науки
- Научный проект требует чёткой структуры и контроля версий данных
- Ручное управление файлами симуляций приводит к ошибкам
- DrWatson решает эти проблемы на уровне экосистемы Julia

Цель работы

- Освоить пакет **DrWatson** для организации воспроизводимых проектов

Цель работы

- Освоить пакет **DrWatson** для организации воспроизводимых проектов
- Изучить функции навигации, сохранения и сбора результатов

Цель работы

- Освоить пакет **DrWatson** для организации воспроизводимых проектов
- Изучить функции навигации, сохранения и сбора результатов
- Автоматизировать сборку отчёта с помощью Makefile и Quarto

Задачи

- Установить DrWatson и инициализировать проект `MyProject`

Задачи

- Установить DrWatson и инициализировать проект `MyProject`
- Проверить функции `datadir()`, `plotsdir()`, `projectdir()`

Задачи

- Установить DrWatson и инициализировать проект `MyProject`
- Проверить функции `datadir()`, `plotsdir()`, `projectdir()`
- Сохранить результаты через `savename` + `safesave`

Задачи

- Установить DrWatson и инициализировать проект `MyProject`
- Проверить функции `datadir()`, `plotsdir()`, `projectdir()`
- Сохранить результаты через `savename` + `safesave`
- Запустить пакетные симуляции через `dict_list`

Задачи

- Установить DrWatson и инициализировать проект `MyProject`
- Проверить функции `datadir()`, `plotsdir()`, `projectdir()`
- Сохранить результаты через `savename` + `safesave`
- Запустить пакетные симуляции через `dict_list`
- Собрать результаты в DataFrame через `collect_results`

Задачи

- Установить DrWatson и инициализировать проект `MyProject`
- Проверить функции `datadir()`, `plotsdir()`, `projectdir()`
- Сохранить результаты через `savename` + `safesave`
- Запустить пакетные симуляции через `dict_list`
- Собрать результаты в DataFrame через `collect_results`
- Создать Makefile и шаблон проекта через PkgTemplates

Материалы и методы

- Язык программирования: Julia

Материалы и методы

- Язык программирования: `Julia`
- Пакеты: `DrWatson`, `PkgTemplates`, `DataFrames`, `BSON`, `JLD2`

Материалы и методы

- Язык программирования: `Julia`
- Пакеты: `DrWatson`, `PkgTemplates`, `DataFrames`, `BSON`, `JLD2`
- Автоматизация: `Makefile`

Материалы и методы

- Язык программирования: `Julia`
- Пакеты: `DrWatson`, `PkgTemplates`, `DataFrames`, `BSON`, `JLD2`
- Автоматизация: `Makefile`
- Рендеринг отчёта: `Quarto`

Раздел 2

Теоретическая часть

Что такое DrWatson

- Пакет Julia для организации воспроизводимых научных проектов

Что такое DrWatson

- Пакет Julia для организации воспроизводимых научных проектов
- Стандартизирует структуру директорий

Что такое DrWatson

- Пакет Julia для организации воспроизводимых научных проектов
- Стандартизирует структуру директорий
- Предоставляет переносимые пути к файлам

Что такое DrWatson

- Пакет Julia для организации воспроизводимых научных проектов
- Стандартизирует структуру директорий
- Предоставляет переносимые пути к файлам
- Автоматизирует именование и сохранение данных

Что такое DrWatson

- Пакет Julia для организации воспроизводимых научных проектов
- Стандартизирует структуру директорий
- Предоставляет переносимые пути к файлам
- Автоматизирует именование и сохранение данных
- Поддерживает сбор результатов из множества файлов

Структура проекта DrWatson

```
MyProject/
├── Project.toml      # Зависимости
├── Manifest.toml     # Точные версии
├── data/
│   ├── sims/         # Результаты симуляций
│   ├── exp_raw/      # Сырые данные
│   └── exp_pro/      # Обработанные данные
├── scripts/          # Скрипты анализа
├── src/              # Исходный код
└── _research/        # Черновики
```

Ключевые функции DrWatson

Функция	Назначение
`datadir()`	Путь к `data/`
`plotsdir()`	Путь к `plots/`
`projectdir()`	Корень проекта
`savename(p, ext)`	Имя файла по параметрам
`safesave(path, d)`	Сохранение без перезаписи
`collect_results(dir)`	Сбор в DataFrame
`dict_list(d)`	Декартово произведение

Раздел 3

Выполнение работы

Установка и компиляция зависимостей

```
 [8e850ede] + nghttp2_jll v1.64.0+1
 [3f19e933] + p7zip_jll v17.7.0+0
 ✓ JLD2
 ✓ JLD2 → UnPackExt
recompiling packages finished.
 2 dependencies successfully precompiled in 94 seconds. 43 already precompiled.
 2 dependencies precompiled but different versions are currently loaded. Restart julia to access the new
versions. Otherwise, loading dependents of these packages may trigger further precompilation to work wit
 the unexpected versions.
MyProject"
```

Рисунок 1: Компиляция зависимостей: nghttp2_jll, p7zip_jll, JLD2

Проверка функций навигации

```
julia> datadir()
"/home/aifedorova/work/study/2026-1/2026-1--study-mathmod/labs/lab04/MyProject/data"

julia> plotsdir()
"/home/aifedorova/work/study/2026-1/2026-1--study-mathmod/labs/lab04/MyProject/plots"

julia> projectdir()
"/home/aifedorova/work/study/2026-1/2026-1--study-mathmod/labs/lab04/MyProject"

julia> scriptdir("analysis.jl")
ERROR: UndefVarError: `scriptdir` not defined in `Main`
Suggestion: check for spelling errors or missing imports.
Stacktrace:
 [1] top-level scope
   @ REPL[10]:1
```

Рисунок 2: datadir(), plotsdir(), projectdir() и ошибка scriptdir

Замечание: scriptdir vs scriptsdir

- В DrWatson функция называется **`scriptsdir()`** (с **`s`** на конце)

Замечание: scriptdir vs scriptsdir

- В DrWatson функция называется **`scriptsdir()`** (с **`s`** на конце)
- `scriptdir` — несуществующая функция → `UndefVarError`

Замечание: scriptdir vs scriptsdir

- В DrWatson функция называется **`scriptsdir()`** (с **`s`** на конце)
- `scriptdir` — несуществующая функция → `UndefVarError`
- Все функции навигации DrWatson используют форму во множественном числе: `datadir`, `plotsdir`, `scriptsdir`, `srcdir`

Сохранение результатов симуляции

```
julia> using DrWatson: savename, @dict, safesave

julia> params = @dict(α=0.1, β=2.0)
Dict{Symbol, Float64} with 2 entries:
  :α => 0.1
  :β => 2.0

julia> results = Dict(:data => rand(10))
Dict{Symbol, Vector{Float64}} with 1 entry:
  :data => [0.894939, 0.35093, 0.0206195, 0.0473639, 0.880771, 0.0660871, 0.752385, 0.521048, 0.0145327,…

julia> filename = savename(params, "jld2")
"α=0.1_β=2.0.jld2"

julia> safesave(datadir("sims", filename), results)
┌ Warning: you passed a key as a symbol instead of a string, it has been automatically converted
└ @ JLD2 ~/.julia/packages/JLD2/hbsZG/src/fileio.jl:10
```

Рисунок 3: savename + safesave: автоматическое именование файла

Как работает savename

```
params = @dict(α=0.1, β=2.0)
filename = savename(params, "jld2")
# → "α=0.1_β=2.0.jld2"

results = Dict(:data => rand(10))
safesave(datadir("sims", filename), results)
```

- `@dict` создаёт словарь с именами переменных как ключами

Как работает savename

```julia
params = @dict(α=0.1, β=2.0)
filename = savename(params, "jld2")
# → "α=0.1_β=2.0.jld2"

results = Dict(:data => rand(10))
safesave(datadir("sims", filename), results)
```

- `@dict` создаёт словарь с именами переменных как ключами
- `savename` формирует уникальное имя файла

Как работает savename

```julia
params = @dict(α=0.1, β=2.0)
filename = savename(params, "jld2")
# → "α=0.1_β=2.0.jld2"

results = Dict(:data => rand(10))
safesave(datadir("sims", filename), results)
```

- `@dict` создаёт словарь с именами переменных как ключами
- `savename` формирует уникальное имя файла
- `safesave` не перезаписывает существующие файлы

Пакетная обработка параметров

```
  1 dependency successfully precompiled in 5 seconds

julia> using BSON

julia> all_params = Dict(:α => [0.1, 0.5], :β => 1:5)
Dict{Symbol, AbstractVector} with 2 entries:
  :α => [0.1, 0.5]
  :β => 1:5

julia> for params in dict_list(all_params)
           result = simulate(params)
           safesave(datadir("sims", savename(params, "bson")), result)
       end
ERROR: UndefVarError: `simulate` not defined in `Main`
Suggestion: check for spelling errors or missing imports.
Stacktrace:
 [1] top-level scope
   @ ./REPL[24]:2
```

Рисунок 4: Ошибка: функция simulate не определена в Main

Решение: модуль в src/

```julia
# src/MyProject.jl
module MyProject
export simulate

function simulate(params)
    # логика симуляции
end
end
```

- Пользовательские функции выносятся в `src/`

Решение: модуль в src/

```julia
# src/MyProject.jl
module MyProject
export simulate

function simulate(params)
    # логика симуляции
end
end
```

- Пользовательские функции выносятся в `src/`
- Подключаются через `using MyProject`

Решение: модуль в src/

```julia
# src/MyProject.jl
module MyProject
export simulate

function simulate(params)
    # логика симуляции
end
end
```

- Пользовательские функции выносятся в `src/`
- Подключаются через `using MyProject`
- Это стандартная практика DrWatson

Сбор результатов в DataFrame

```
Julia
  @ ./REPL[24]:2

julia> using DataFrames

julia> ## Сбор всех данных в DataFrame
       df = collect_results(datadir("sims"))

[ Info: Scanning folder /home/aifedorova/work/study/2026-1/2026-1--study-mathmod/labs/lab04/MyProject/dat
a/sims for result files.
[ Info: Added 1 entries.
1×2 DataFrame
 Row │ data                                path
     │ Array…?                             String?
─────┼──────────────────────────────────────────────────────────────────────────────────────────────────
   1 │ [0.894939, 0.35093, 0.0206195, 0…  /home/aifedorova/work/study/2026…

julia> select!(df, :α, :β, :result)
ERROR: ArgumentError: column name :α not found in the data frame
Stacktrace:
 [1] lookupname
   @ ~/.julia/packages/DataFrames/b4w9K/src/other/index.jl:431 [inlined]
 [2] getindex
   @ ~/.julia/packages/DataFrames/b4w9K/src/other/index.jl:440 [inlined]
 [3] normalize_selection(idx::DataFrames.Index, sel::Symbol, renamecols::Bool)
   @ DataFrames ~/.julia/packages/DataFrames/b4w9K/src/abstractdataframe/selection.jl:382
```

Почему нет столбцов α и β

- `safesave` сохранил только ключи из `results`

```
safesave(path, merge(params, results))
# или использовать tagsave()
```

Почему нет столбцов α и β

- `safesave` сохранил только ключи из `results`
- Параметры α, β не вошли в файл

```
safesave(path, merge(params, results))
# или использовать tagsave()
```

Почему нет столбцов α и β

- `safesave` сохранил только ключи из `results`
- Параметры α, β не вошли в файл
- **Правильно:** объединять параметры с результатами:

```
safesave(path, merge(params, results))
# или использовать tagsave()
```

Makefile для автоматизации

```
aifedorova@aifedorova: ~/work/study/2026-1/2026-1--study-mathmod/labs/lab04/MyProject
GNU nano 7.2                                   Makefile *
.PHONY: report clean

report:
    julia --project=. scripts/run_simulations.jl
    julia --project=. scripts/generate_report.jl
    quarto render docs/report.qmd

clean:
    rm -rf data/sims/*
    rm -rf plots/*

                                   [ Read 10 lines ]
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute     ^C Location    M-U Undo
```

Структура Makefile

```
.PHONY: report clean

report:
    julia --project=. scripts/run_simulations.jl
    julia --project=. scripts/generate_report.jl
    quarto render docs/report.qmd

clean:
    rm -rf data/sims/*
    rm -rf plots/*
```

Создание проекта через PkgTemplates

aifedorova@aifedorova: ~/work/study/2026-1/2026-1--study-mathmod/labs/lab04/MyProject

```
  GNU nano 7.2                                  src/MyProject.jl *
# src/MyProject.jl
module MyProject
export simulate

function simulate(params)
  # ...
end
end
^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
```

Шаблон через PkgTemplates

```julia
using PkgTemplates
t = Template(;
    dir="~/Projects",
    julia=v"1.9",
    plugins=[DrWatsonPlugin()]
)
t("MyNewProject")
```

- Генерирует полную структуру проекта DrWatson

Шаблон через PkgTemplates

```julia
using PkgTemplates
t = Template(;
    dir="~/Projects",
    julia=v"1.9",
    plugins=[DrWatsonPlugin()]
)
t("MyNewProject")
```

- Генерирует полную структуру проекта DrWatson
- Настраивает целевую версию Julia

Шаблон через PkgTemplates

```julia
using PkgTemplates
t = Template(;
    dir="~/Projects",
    julia=v"1.9",
    plugins=[DrWatsonPlugin()]
)
t("MyNewProject")
```

- Генерирует полную структуру проекта DrWatson
- Настраивает целевую версию Julia
- Подключает нужные плагины автоматически

Раздел 4

Результаты

Рабочий процесс DrWatson

1. `initialize_project()` → стандартная структура директорий

Рабочий процесс DrWatson

1. `initialize_project()` → стандартная структура директорий
2. `@quickactivate` → активация окружения проекта

Рабочий процесс DrWatson

1. `initialize_project()` → стандартная структура директорий
2. `@quickactivate` → активация окружения проекта
3. `savename()` + `safesave()` → именованное сохранение данных

Рабочий процесс DrWatson

1. `initialize_project()` → стандартная структура директорий
2. `@quickactivate` → активация окружения проекта
3. `savename()` + `safesave()` → именованное сохранение данных
4. `dict_list()` → пакетные симуляции по параметрам

Рабочий процесс DrWatson

1. `initialize_project()` → стандартная структура директорий
2. `@quickactivate` → активация окружения проекта
3. `savename()` + `safesave()` → именованное сохранение данных
4. `dict_list()` → пакетные симуляции по параметрам
5. `collect_results()` → сбор результатов в DataFrame

Рабочий процесс DrWatson

1. `initialize_project()` → стандартная структура директорий
2. `@quickactivate` → активация окружения проекта
3. `savename()` + `safesave()` → именованное сохранение данных
4. `dict_list()` → пакетные симуляции по параметрам
5. `collect_results()` → сбор результатов в DataFrame
6. Makefile + Quarto → автоматический рендеринг отчёта

Выявленные ошибки и их решения

- `scriptdir` → правильно: **`scriptsdir()`**

Выявленные ошибки и их решения

- `scriptdir` → правильно: **`scriptsdir()`**
- `simulate` не определена → выносить в **`src/MyProject.jl`**

Выявленные ошибки и их решения

- `scriptdir` → правильно: `scriptsdir()`
- `simulate` не определена → выносить в `src/MyProject.jl`
- Столбцы параметров не в DataFrame → использовать `merge(params, results)` или `tagsave`

Выводы

- DrWatson обеспечивает воспроизводимость через стандартизацию структуры

Выводы

- DrWatson обеспечивает воспроизводимость через стандартизацию структуры
- `savename` + `safesave` исключают случайную перезапись данных

Выводы

- DrWatson обеспечивает воспроизводимость через стандартизацию структуры
- `savename` + `safesave` исключают случайную перезапись данных
- `collect_results` автоматизирует сбор результатов из множества файлов

Выводы

- DrWatson обеспечивает воспроизводимость через стандартизацию структуры
- `savename` + `safesave` исключают случайную перезапись данных
- `collect_results` автоматизирует сбор результатов из множества файлов
- Makefile + Quarto образуют полный пайплайн от симуляций до отчёта

Выводы

- DrWatson обеспечивает воспроизводимость через стандартизацию структуры
- `savename` + `safesave` исключают случайную перезапись данных
- `collect_results` автоматизирует сбор результатов из множества файлов
- Makefile + Quarto образуют полный пайплайн от симуляций до отчёта
- PkgTemplates позволяет тиражировать структуру проекта